

PROJECT AND TEAM INFORMATION

Project Title

8-Bit Emulator — A Simple CPU Simulation using C/C++ and Data Structures in C

Student / Team Information

<i>Team Name:</i> <i>Team #</i>	<i>NULL Pointers</i>
Team member 1 (Team Lead)	<i>Bisht, Arnav – 2510011057 2510011057@geu.ac.in</i> 
Team member 2	<i>Shukla, Piyush – 2510380136 2510380136@geu.ac.in</i> 
Team member 3	<i>Sharma, Ashmit – 2510371320 2510371320@geu.ac.in</i> 
Team member 4	<i>Singh, Aashish – 2510370547 2510370547@geu.ac.in</i> 

PROPOSAL DESCRIPTION

Motivation

Modern computer systems run programs by following a planned sequence of actions that include registers, memory, instructions, control flow and the state of the processor. These ideas are very important in computer science. They can be hard to imagine because a regular programming setup keeps the inner workings of a CPU hidden behind compilers, operating systems and hardware. This project tries to fix that by creating a 8-bit processor that is completely made in software so the inside workings of a CPU can be seen and changed directly.

The emulator that is planned will have its simple set of instructions, registers, memory system, flags and rules for how things work. A person will be able to write a program in a style similar to assembly or load one in and watch as each instruction goes through the steps of getting it understanding it and running it. After each instruction the changes to the registers memory, the program counter, the stack and the processor flags can be checked of being kept secret inside the machine.

In addition to showing how computer architecture works the project uses examples of important C/C++ ideas. Things, like arrays, stacks, queues, strings, searching, breaking down code, structures and handling files can be used in the emulator. C++ classes can help put the CPU, memory, instructions and the part that runs everything into parts. The end result is a system that works without needing an internet connection and can be used over and over again where both the way computers are built and how data structures are used can be studied through one working model. The goal of this project matches the plans focus on making what happens inside the CPU visible and using data structures in a real system.

State of the Art / Current solution

Real processors like ARM, x86 and AVR rely on hardware to carry out instruction execution. Full-system emulators try to mimic these processors. They come with a lot of complexity. These kinds of systems are usually too advanced for a beginner who wants to learn the basics of CPU design and data structures.

Educational CPU simulators make things easier, by simplifying the processor architecture.. Often the inner workings stay hidden behind the user interface. Our emulator takes an approach. It is built with transparency. We implement the instruction representation, parser, memory, stack and execution engine directly in C or C++.

This way users can see how the simulator works from the inside. They can learn not how a CPU runs instructions but also how the simulator itself is constructed. It's a hands-on way to understand both CPU behavior and software design.

Project Goals and Milestones

The main goal is to create a working 8-bit CPU that can load, understand and run programs written in a special kind of assembly language. The system will include registers, memory that can be accessed by bytes, processor flags, a program counter and a stack. The main goal is to create a working 8-bit CPU that can load, understand and run programs written in a special kind of assembly language. The system will include registers, memory that can be accessed by bytes, processor flags, a program counter and a stack.

It will handle math operations, logic operations, loading and storing data and jumping between parts of a program. It will also have a process that fetches, decodes and runs each instruction. The project will include running step, by step watching the state of the processor keeping a record of what happened during execution and saving programs in files. The implementation will use C data structures and algorithms. The emulator parts will be organized using C++ classes.

Milestones

M1. Review C++ and data structures and algorithms concepts; complete the design of PIXEL-8 architecture, CPU design and instruction set.

M2. Set up memory, CPU registers, program counter, stack pointer and basic system state.

M3. Create the way instructions are shown look up opcodes build a parser and check operands.

M4. Put into action the fetch-decode-execute cycle and the main arithmetic, logical and memory instructions.

M5. Include stack and queue features, branching, CALL and RETURN actions and changes in system state.

M6. Build a 2D frame buffer handle pixel and tile graphics and accept input from keyboard or controller.

M7. Put in an original game like Pixel Dodge and add controls for start, pause, reset and exit.

M8. Do testing, fix errors, write documentation check performance and do the demonstration.

Project Approach

C — Data Structures & Algorithms

The C implementation will rely on data structures to handle the emulator's inner workings. Arrays will hold registers and memory contents. A stack will manage PUSH and POP actions, well as support subroutine calls. Queues can track input or events that need processing. Linked lists might be used for tracking execution details that change over time.

String handling and tokenization will break down assembly-style instructions into opcodes and operands. Searching will allow access to opcodes or symbols during execution. Sorting could help keep memory or execution data in order when needed. These structures are built directly into the emulator not just added for example purposes. They serve functions, in how the system operates.

C++ — OOP Design

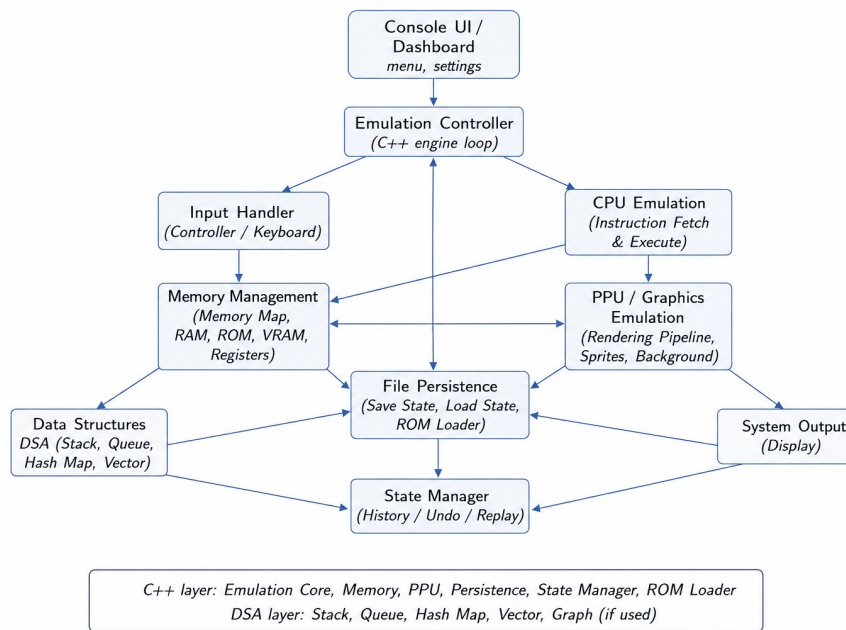
The C++ code will split the emulator into classes that work together. A CPU class will take care of registers, flags and the program counter while a Memory class will handle memory tasks. An Instruction class will hold the decoded commands and their values and an Emulator or ExecutionEngine class will control the process of fetching, decoding and running instructions. A Program class will manage the loaded code and the instructions it contains.

Encapsulation will help keep each parts information and actions separate making the whole system easier to build and check. Constructors, destructors, inheritance and polymorphism can be used where needed to show object-oriented programming ideas while keeping the design simple and easy to work with.

Platform & Tools

The project will use C11 and C++17 programming languages. It will be compiled using GCC and G++ on either Windows or Linux operating systems. For development the tools will be VS Code or CodeBlocks. For version control and collaboration the tools will be Git and GitHub. A Makefile might be used for compilation. Text files or CSV files will store programs and execution logs. The tool draw.io will be used for creating system diagrams and class diagrams. The emulator will mostly have a console-based interface. Testing will include execution as well as boundary cases such, as overflow, invalid opcodes, illegal memory access and incorrect branch targets.

System Architecture (High Level Diagram)



Project Outcome / Deliverables

The final deliverable will be a working menu-driven 8-bit CPU emulator. This 8-bit CPU emulator will. Run programs that use the project's own assembly-like instruction set. The 8-bit CPU emulator will perform a fetch-decode-execute cycle. It will keep track of processor registers, memory, flags, program counter and stack state. All of these components will work together to make the system function properly.

The system will provide a step-by-step debugging environment for users.

In this debugging environment users can see processor state after each instruction. Observe logical, memory, branching and stack operations in sample programs.

Program files and execution logs will be saved persistently so that the same programs can be loaded and tested again and again.

The project will give C/C++ source code, an instruction-set specification, system architecture and class diagrams. The project will also give documentation on algorithms and complexity a user guide and a README file. The final demonstration will show the workflow—from loading a program to executing a program step by step—while letting observers see the state of the virtual processor.

Assumptions

The proposed emulator is a simplified processor; not an exact copy of any commercial CPU. The emulator will have a fixed number of registers a set memory capacity, specific instruction formats and a defined set of supported operations, all of which will be laid out in the project specification. Programs for the emulator can be entered manually. Loaded from text files and the emulator will run entirely in software without the need, for any physical hardware.

All arithmetic operations will follow the defined 8-bit representation. Processor flags will be used to show conditions such, as zero, carry and other status conditions that the instruction set supports. The system is meant for a local user. The system will value transparency and educational value over performance. Invalid instructions, memory addresses stack errors and other boundary conditions will be detected and reported explicitly instead of causing undefined emulator behavior.

References

1. *David A. Patterson & John L. Hennessy, Computer Organization and Design — processor organization, instruction execution and memory concepts.*
2. *Bjarne Stroustrup, The C++ Programming Language, 4th ed. — classes, inheritance, polymorphism and resource management.*
3. *Kernighan & Ritchie, The C Programming Language, 2nd ed. — arrays, pointers, structures and dynamic memory.*
4. *E. W. Dijkstra, “A note on two problems in connexion with graphs,” Numerische Mathematik, 1959 — algorithmic background where applicable.*
5. *GCC Documentation — compiling and testing C/C++ programs.*
6. *Project-specific instruction-set and architecture specification to be finalized by the team.*